

Evaluation & Operations — Turning Traces Into a Ratchet

Track 16.5 — LangSmith · 16-ai-tech-stack/tracks/langsmith/notes/02-evaluation-and-operations.md

Note 01 gave you the trace. This note is about what you do with it: build datasets, score them, gate CI on the result, put humans in the loop, and watch production. The *theory* of evaluation — benchmark design, contamination, inter-annotator agreement, metric validity — lives in `../15-ai-evals/`. **This note is the tooling.** Read that module first if you have not; a well-plumbed eval of the wrong thing is still the wrong thing.

1. Datasets: production traces are the only good source

An **example** is `inputs` + optional `reference_outputs` + optional `metadata` + optional `split`. A **dataset** is a list of examples with automatic versioning.

Where teams get examples, ranked worst to best:

Source	Verdict
Invented in a planning meeting	Reflects what you <i>imagine</i> users ask. Always wrong, in the same direction.
Copied from a public benchmark	Measures general capability, not your app. Probably in the training data anyway.
LLM-generated synthetic cases	Useful for <i>coverage</i> of edge shapes; will not surface real-world weirdness. Fine as a supplement.
Your own dogfooding traces	Good. This is how you start on day zero with no users.
Production traces that went wrong	The answer.

The filters that harvest a dataset worth having:

- `feedback.user_score = negative` — users telling you directly
- `error = true` — tool timeouts, schema failures, parse errors
- `latency > p99` — the shape of a failure you cannot see in the text
- `online_judge_score < 0.5` — automated triage over live traffic (§5)
- `metadata.customer_tier = enterprise` — the failures that cost the most

In the UI that is: filter a tracing project, multi-select runs, **Add to Dataset**. In the SDK it is `client.create_dataset` / `client.create_examples`. Either way, **carry the source run id in the example's metadata**, so six months from now you can still open the trace that motivated the case.

Say it out loud: your eval set should be a museum of your own outages. If you cannot point at the incident that produced an example, ask why the example is there.

Splits and versions

Splits are named subsets of the same dataset — `smoke` (10 cases, runs on every commit, under a minute), `regression` (a few hundred, runs on every PR), `adversarial` (the red-team set from `../../13-ai-security/`). An example can be in several splits.

Versions are automatic: every add/update/delete creates a new dataset version, and you can tag one. Tag the version CI pins to (`client.update_dataset_tag(..., tag="ci-pinned")`). Otherwise a teammate adding twenty hard examples on Tuesday reads in your dashboard as "the model got worse on Tuesday", and you will spend a day chasing it.

2. `evaluate()` — the API

```
from langsmith import evaluate  # aevaluate() is the async twin, same interface

def target(inputs: dict) -> dict:  # your app under test
    return {"answer": support_agent(inputs["question"])}

def correct(inputs: dict, outputs: dict, reference_outputs: dict) -> dict:
    return {"key": "correct", "score": float(outputs["answer"] == reference_outputs["answer"])}

evaluate(
    target,
    data="support-regression",  # dataset name, UUID, or example iterator
    evaluators=[correct, groundedness],
    experiment_prefix="prompt-v7",
    max_concurrency=8,
    num_repetitions=3,
    metadata={"git_sha": GIT_SHA},
)
```

Four details that matter more than they look:

- **The evaluator signature is keyword-matched.** Declare the subset you need (`inputs`, `outputs`, `reference_outputs`, `run`, `example`) and the SDK passes it. Return `{"key", "score"}` plus an optional `comment` — put your reasoning in `comment`, because in three weeks a bare `0.4` means nothing.
- **`num_repetitions` is not optional in spirit.** Your system is nondeterministic. One pass over a dataset is an anecdote. Three passes give you a variance estimate, and variance is what tells you whether your +2% is real.
- **`metadata={"git_sha": ...}` is how an experiment becomes traceable to a commit.** Do it from the start.
- **Every evaluation produces traces too.** An experiment is runs-with-scores. When one example fails you open its trace like any other. This is the actual reason to co-locate evals with tracing rather than running them in a notebook.

A runnable, network-free version of this whole loop — dataset, evaluators, aggregation, baseline comparison, CI gate — is `../code/eval_harness.py`.

Evaluator taxonomy

Kind	Example	Cost	Use for
Deterministic	exact match, JSON-schema valid, regex, contains-citation	~free	structured output, hard constraints
Statistical	embedding similarity, ROUGE, BLEU	cheap	rough drift signals only
Programmatic domain check	SQL executes and returns the right rows; code passes its tests	cheap	anything with ground truth you can <i>run</i>
LLM-as-judge	groundedness, helpfulness, tone, rubric score	expensive	subjective quality, no reference available
Human	annotation queue with a rubric	very expensive	calibration, high-stakes, judge validation

Reach for the cheapest kind that actually measures the thing. Teams jump straight to LLM-as-judge for problems a five-line assertion would settle, then spend a month debugging the judge.

3. LLM-as-judge, and the three biases that will burn you

A judge is an LLM plus a rubric. **The rubric is the product.** "Rate this answer 1-10" produces a number with no construct validity; a rubric with named, checkable criteria and a stated scale produces something you can argue with.

```
BAD:  "Rate the helpfulness of this answer from 1 to 10."

GOOD: "Score each criterion independently. Output JSON.
      grounded: every factual claim appears in CONTEXT      (0 or 1)
      complete: addresses every part of the question        (0 or 1)
      concise:  no preamble, no restating the question      (0 or 1)
      safe:     no promises beyond the published policy     (0 or 1)
      Then: reasoning (<=2 sentences), then the JSON object."
```

Binary criteria beat 1–10 scales. Models cannot reliably distinguish a 6 from a 7, and neither can your annotators, so you are averaging noise. Ask for reasoning *before* the score — it measurably improves judgments and, more usefully, lets you read *why* when you disagree.

The biases

Bias	What it looks like	Mitigation
Position bias	Given A then B, the judge prefers A — on identical content	<code>randomize_order=True</code> ; or score both orders and average
Verbosity bias	Longer reads as more helpful, regardless of correctness	Explicit concision criterion; check score-vs-length correlation
Self-preference	A model rates its own family higher	Judge with a <i>different</i> family than the one you are scoring

Two more worth knowing: **leniency drift** (judges cluster at the top of any scale — most of your scores will be 0.8+, which destroys resolution) and **format bias** (bullet points score above prose).

`eval_harness.py` reproduces verbosity bias deterministically, with no API key: a naive length-sensitive judge scores a padded answer **+0.61** over the baseline, while a criterion-level rubric shows the real effect is a **+0.11 coverage win and a -0.41 concision regression** that nearly cancel. Watch it happen, then never trust a single composite number again.

Calibrating a judge (the step everyone skips)

1. Take 50–100 examples. Have a **human** label them with your rubric.
2. Run the judge on the same examples.
3. Compute agreement (Cohen's kappa, or just percent-agreement per criterion).
4. Below ~ 0.7 agreement, the judge is not measuring your rubric. Fix the *prompt*, not the model.
5. Re-check quarterly, and after any model upgrade. A judge is a model, and models change.

A judge that has never been checked against a human is a random number generator with good manners.

4. Pairwise comparison — for when absolute scoring is hopeless

"Is this answer an 0.7 or an 0.8?" is unanswerable by humans and models alike. "Is A better than B?" is easy for both. That is the entire case for pairwise.

```
from langsmith import evaluate_comparative

def prefer(inputs: dict, outputs: list[dict]) -> list[float]:
    return [1, 0] if better(outputs[0], outputs[1]) else [0, 1]

evaluate_comparative(
    ["prompt-v6-a1b2c3", "prompt-v7-d4e5f6"],  # two EXISTING experiments
    evaluators=[prefer],
    randomize_order=True,                      # position-bias mitigation, on by design
)
```

Use pairwise for: prompt A/B, model migration (`../../11-llm-models/`), tone and style, and any "does this feel better" question. Do **not** use it for hard constraints — "did it leak a credit card number" is pass/fail, not a preference, and averaging a constraint into a quality score is how constraints get shipped broken.

The limitation: pairwise gives you a *relative* answer. Both candidates can be terrible. Keep at least one absolute floor metric alongside it.

5. Online evaluation and monitoring — the production half

Offline evaluation needs reference outputs. Production has none. **Online evaluation** runs reference-free evaluators over live traces: groundedness, policy compliance, refusal rate, tone, "did it answer the question at all".

In LangSmith this is configured per tracing project under **Evaluators**, with three knobs that matter:

- **A filter** — which runs trigger it (`feedback = thumbs down`, `tool = search`, `metadata.tier = enterprise`). Judge the runs you care about.
- **A sampling rate** — the percentage of *filtered* runs that fire. Judging 100% of production means every request costs you two inferences. 5–10% of healthy traffic plus 100% of anything with negative feedback is a reasonable starting shape.
- **A spend limit** — an actual weekly cap. Set it. An online judge is an unbounded spend loop attached to your traffic, and traffic spikes.

Rules/automations can also route matching runs straight into a **dataset** or an **annotation queue**, or fire a **webhook**. That is the loop from note 01, automated: a production failure becomes a test case without anyone remembering to do it.

Alerting. Alert on *derived quality and cost*, not just errors: p95 cost per trace, p95 latency, error-run rate, negative-feedback rate, and online-judge score. Route them to the same place as your ordinary SRE alerts — a quality dashboard nobody has a pager for is a dashboard nobody reads.

6. Human feedback — the ground truth everything else is calibrated against

Three channels, in ascending order of signal quality and cost:

1. **Implicit.** Did the user retry, rephrase, copy the answer, abandon the session? Free, noisy, and surprisingly predictive. Log it as feedback on the run.
2. **Explicit thumbs up/down.** Return the `run_id` to your frontend, POST it back with the vote:

```
python  
run_id = str(get_current_run_tree().id) # inside the traced function  
client.create_feedback(run_id=run_id, key="user_score", score=0.0, comment=note)
```

 Plumb this in **week one**. It is thirty lines and it is the difference between "users seem unhappy" and a filterable, dataset-

able signal. Expect a low single-digit percentage response rate, heavily skewed negative — that skew is a feature, since negatives are what you want.

3. **Annotation queues.** Structured human review: a queue with reviewer instructions and a categorical rubric, items routed in by hand (up to 100 at a time), from an experiment, or automatically by a rule. Reviewers score against the rubric, leave notes, and **Add to Dataset** in one click. Options that matter: multiple reviewers per item (for agreement measurement), reservations (so two people don't review the same item), and a **pairwise** queue mode for side-by-side experiment comparison.

The pipeline to actually build: *thumbs-down* → *rule routes it to an annotation queue* → *human confirms it is really wrong and writes the correct answer* → *Add to Dataset* → *next CI run blocks the regression*. That chain, end to end, is the difference between a team that improves and a team that ships vibes.

7. Prompts: versioning without a deploy

LangSmith stores prompts with git-like semantics: every save is a **commit** with a hash, commits take human-readable **tags**, and `staging` / `production` are reserved environment tags.

```
prompt = client.pull_prompt("my-org/support-system:production")
prompt = client.pull_prompt("my-org/support-system:a1b2c3d") # pin a commit
```

Worth it when non-engineers edit prompts and you still need rollback and an audit trail (`../../../../14-ai-compliance/` will ask for one), or when you want to iterate in the Playground against real traced inputs.

Not worth it when prompts are code owned by engineers. Git already does versioning, review and rollback, and a prompt that can change without a deploy is a prompt that can break production without a deploy. If you do use the hub in production, pin a **commit hash or an environment tag**, never a bare name — otherwise someone's Playground experiment is a live config change.

8. Self-hosting and data residency

- **Cloud** — managed, with US / EU / APAC data regions selected via `LANGSMITH_ENDPOINT`.
- **BYOC** — runs in your VPC, LangChain operates it. Enterprise plan.
- **Self-hosted** — entirely your infrastructure. Enterprise plan.

If you are in a regulated shop, start the self-hosting conversation **before** you instrument, because the answer changes your architecture and your contract. If self-hosting is non-negotiable and the budget is not, that constraint alone points at Langfuse or Phoenix — see the table below.

9. Alternatives — choose X when

No sponsorship here. LangSmith is a good product with a real cost and a real vendor relationship.

Tool	Model	Strength	Weakness	Choose it when
LangSmith	Hosted SaaS; self-host on Enterprise	Best-integrated trace→dataset→eval→annotate loop; automatic for LangChain/LangGraph; strong annotation queues	Per-seat/volume cost; self-host is Enterprise-gated; ecosystem gravity	You want one tool for the whole loop and you are shipping, not evaluating vendors
Langfuse	Open source, self-hostable (Postgres + ClickHouse)	Genuinely free to self-host; framework-agnostic; OTEL-friendly; good prompt management	Eval tooling is thinner; you operate the database	Self-hosting is a hard requirement, or the seat bill is the blocker
Arize Phoenix	Open source, OTEL-native; commercial Arize AX above it	Runs in a notebook or a container; strongest embedding/drift analysis; clean OTEL story	Less of an end-to-end product; team workflow features are lighter	You are OTEL-first, or doing embedding/drift analysis on a live system
Braintrust	Hosted, self-host available	Eval-first design; strong CI-gated release workflow; generous free tier	Younger ecosystem; tracing is good but eval is the centre of gravity	Evals are the main event and you want them wired into release gating
OTel + GenAI semantic conventions	Open standard	No lock-in; one pipeline for app and LLM telemetry; your existing backend	Conventions still stabilising; you build datasets/judges/annotation yourself	You already run serious observability and refuse a second vendor
Plain structured logging	Free	Zero dependencies; works today	No tree view, no diffing, no dataset, no eval; you <i>will</i> rebuild a worse version of the above	Prototype, or a single non-agentic LLM call in a bigger app

A defensible default: **instrument with OpenTelemetry semantics, ship to LangSmith or Langfuse.**

LangSmith accepts OTLP and can export to a collector, so this is not hypothetical. You get the product now and an exit later, which is the only kind of vendor bet worth making.

And the honest baseline: **for one non-agentic LLM call, plain logging is fine.** The moment you have a chain, a tool, or a retriever — three spans and a loop — the tree view pays for itself in one debugging session. That is the threshold, not team size.

10. The checklist

Before you call an LLM application "in production":

- [] `LANGSMITH_TRACING=true` and traces are visibly arriving in the right project
- [] Release version / git SHA in run metadata
- [] `thread_id` set on multi-turn traffic
- [] Sampling rate chosen deliberately, 100% retention for errors and negative feedback

- [] PII redaction at the SDK boundary, with a named owner for the decision
- [] Thumbs up/down plumbed from the UI to `create_feedback`
- [] A dataset with ≥ 50 examples, every one harvested from a real trace
- [] A `smoke` split that runs in under a minute on every commit
- [] A CI gate with **both** absolute floors and regression deltas against a pinned baseline
- [] At least one LLM-as-judge calibrated against human labels, with the kappa written down
- [] Alerts on p95 latency, p95 cost per trace, error rate, negative-feedback rate
- [] A named human who reads the annotation queue weekly

If the last line is empty, the rest is decoration.